

Complete Guide: Implementing PUF-Based UAV Authentication Protocol in OMNeT++

Executive Summary

This comprehensive implementation guide provides a detailed roadmap for porting your Python-based PUF (Physical Unclonable Function) UAV Authentication Protocol to OMNeT++. The protocol implements a four-phase authentication system for UAV swarm networks, combining PUF hardware primitives with lightweight cryptographic functions like SPONGENT hash and BCH error correction. While no native PUF or SPONGENT libraries currently exist for OMNeT++, this guide presents practical solutions using external C++ libraries, custom implementations, and integration strategies within the INET Framework.^{[1][2][3][4][5]}

Protocol Architecture Overview

Four-Phase Protocol Structure

Your implementation consists of four distinct phases:^{[3][1]}

Phase 1: Enrollment/Registration - Initial UAV setup with Ground Station in a secure environment, where PUF challenge-response pairs (CRPs) are generated and stored with helper data for BCH error correction.

Phase 2: UAV-Ground Station Authentication - UAV authenticates with GS when entering the network, obtaining credentials for peer-to-peer authentication (4 messages, ~210 bytes, 10-50 ms latency).

Phase 3: UAV-to-UAV Peer Authentication - Direct mutual authentication between UAV pairs without GS involvement during flight (3 messages, ~196 bytes, 0.8-1.2 ms latency).

Phase 4: Session Key Establishment - Secure communication channel setup with optional ECDH for perfect forward secrecy.

Core Cryptographic Components

The protocol relies on three essential cryptographic primitives:^{[2][1]}

1. **PUF (Physical Unclonable Function)** - Hardware security primitive generating unique challenge-response pairs; your Python implementation uses Ring Oscillator or Arbiter PUF simulation.
2. **SPONGENT-160 Hash Function** - Lightweight hash function designed for resource-constrained devices, achieving 37% energy reduction compared to AES-based constructions.^[^1]

3. BCH(255, 131, 18) Error Correction - Corrects up to 18 bit errors (~7% BER) in noisy PUF responses, enabling reliable authentication.[²][¹]

OMNeT++ Environment Setup

Installing OMNeT++ and INET Framework

OMNeT++ is an extensible, modular C++ simulation library designed for building network simulators. The INET Framework provides protocol implementations, mobility models, and wireless communication modules essential for UAV networks.[⁶][⁷][⁸]

Installation Steps:

1. Download OMNeT++ 6.0+ from omnetpp.org[⁶]
2. Extract and compile:

```
tar xvfz omnetpp-6.0-linux.tgz
cd omnetpp-6.0
source setenv
./configure
make
```

3. Install INET Framework:

```
git clone https://github.com/inet-framework/inet.git
cd inet
make makefiles
make MODE=release
```

4. Verify installation by running sample simulations in INET/examples.

Project Structure

Create a new OMNeT++ project for your UAV authentication protocol:

```
UAVAuthProtocol/
├── src/
│   ├── crypto/
│   │   ├── PUF Simulator.h/.cc      # PUF simulation module
│   │   ├── SPONGENT.h/.cc          # SPONGENT hash implementation
│   │   ├── BCHCodec.h/.cc          # BCH error correction
│   │   └── CryptoUtils.h/.cc       # XOR, masking utilities
│   ├── nodes/
│   │   ├── UAVNode.h/.cc/.ned      # UAV entity with PUF
│   │   ├── GroundStation.h/.cc/.ned # Ground Station entity
│   │   └── NetworkController.h/.cc  # Simulation controller
│   └── protocols/
│       ├── Phase1Enrollment.h/.cc  # Phase 1 protocol logic
│       ├── Phase2Authentication.h/.cc # Phase 2 protocol logic
│       ├── Phase3PeerAuth.h/.cc    # Phase 3 protocol logic
│       └── Phase4SessionKey.h/.cc   # Phase 4 protocol logic
```

```

├─ simulations/
│   ├── omnetpp.ini          # Configuration file
│   ├── UAVNetwork.ned      # Network topology
│   └─ results/             # Output directory
└─ external/
    ├── libcorrect/         # BCH library
    ├── cryptopp/           # Crypto++ (optional)
    └─ pypuf-cpp/          # Custom PUF simulator

```

Implementing Cryptographic Primitives

PUF Simulation in OMNeT++

Since no native PUF library exists for OMNeT++, you have three implementation options:^{[9][10][11]}

Option 1: Simplified Software PUF Simulator (Recommended for Fast Development)

Create a deterministic but statistically-realistic PUF simulator:

```

// PUF Simulator.h
#ifndef PUFSIMULATOR_H
#define PUFSIMULATOR_H

#include <omnetpp.h>
#include <vector>
#include <random>

using namespace omnetpp;

class PUF Simulator {
private:
    int deviceSeed;           // Unique per UAV
    double noiseLevel;        // BER (e.g., 0.03 for 3%)
    int challengeBits;        // Challenge size (128-bit)
    int responseBits;         // Response size (128-bit)
    std::mt19937 pufRng;      // PUF response generator
    std::mt19937 noiseRng;    // Noise generator

public:
    PUF Simulator(int seed, double noise = 0.03, int challengeSize = 128);

    // Generate PUF response for a challenge
    std::vector<uint8_t> evaluate(const std::vector<uint8_t>& challenge);

    // Simulate Arbiter PUF with delay variations
    int evaluateArbiterBit(const std::vector<bool>& challengeBits);

    // Simulate Ring Oscillator PUF
    int evaluateROPUFBit(int ro1_index, int ro2_index);

    // Add realistic noise to response

```

```

void addNoise(std::vector<uint8_t>& response);

// Calculate inter-chip Hamming distance (uniqueness)
static double calculateUniqueness(const std::vector<uint8_t>& resp1,
                                   const std::vector<uint8_t>& resp2);
};

#endif

```

```

// PUF Simulator.cc
#include "PUFSimulator.h"
#include <string>

PUFSimulator::PUFSimulator(int seed, double noise, int challengeSize)
    : deviceSeed(seed), noiseLevel(noise), challengeBits(challengeSize),
      responseBits(128), pufRng(seed), noiseRng(seed + 1000) {}

std::vector<uint8_t> PUF Simulator::evaluate(const std::vector<uint8_t>& challenge) {
    std::vector<uint8_t> response(responseBits / 8, 0);

    // Seed RNG with challenge and device seed
    uint64_t combinedSeed = deviceSeed;
    for (size_t i = 0; i < challenge.size(); i++) {
        combinedSeed ^= (static_cast<uint64_t>(challenge[i]) << (8 * (i % 8)));
    }
    pufRng.seed(combinedSeed);

    // Generate response bits
    for (int i = 0; i < responseBits; i++) {
        int byteIdx = i / 8;
        int bitIdx = i % 8;

        // Simulate Arbiter PUF delay comparison
        bool bit = (pufRng() % 2 == 0);
        if (bit) {
            response[byteIdx] |= (1 << bitIdx);
        }
    }

    // Add noise
    addNoise(response);

    return response;
}

void PUF Simulator::addNoise(std::vector<uint8_t>& response) {
    int totalBits = responseBits;
    int errorBits = static_cast<int>(totalBits * noiseLevel);

    std::uniform_int_distribution<int> bitDist(0, totalBits - 1);

    for (int i = 0; i < errorBits; i++) {
        int bitPos = bitDist(noiseRng);
        int byteIdx = bitPos / 8;
        int bitIdx = bitPos % 8;
    }
}

```

```

        response[byteIdx] ^= (1 << bitIdx); // Flip bit
    }
}

double PUF Simulator::calculateUniqueness(const std::vector<uint8_t>& resp1,
                                          const std::vector<uint8_t>& resp2) {
    int hammingDistance = 0;
    int totalBits = resp1.size() * 8;

    for (size_t i = 0; i < resp1.size(); i++) {
        uint8_t xorResult = resp1[i] ^ resp2[i];
        // Count set bits
        while (xorResult) {
            hammingDistance += xorResult & 1;
            xorResult >>= 1;
        }
    }

    return static_cast<double>(hammingDistance) / totalBits * 100.0;
}

```

Option 2: Port Python PUF Simulator to C++

Translate your pypuf-based implementation to C++. Extract the Arbiter PUF model from your Python code:

```

# Your Python implementation (from UAV-Authentication repo)
from pypuf.simulation import ArbiterPUF
puf = ArbiterPUF(n=128, seed=42, noisiness=0.03)

```

Convert to C++:

```

class ArbiterPUFModel {
private:
    std::vector<double> delayWeights; // Stage delay variations
    int numStages;

    double computePathDelay(const std::vector<bool>& challenge) {
        double upperDelay = 0.0, lowerDelay = 0.0;
        for (int i = 0; i < numStages; i++) {
            if (challenge[i]) {
                upperDelay += delayWeights[i * 2];
                lowerDelay += delayWeights[i * 2 + 1];
            } else {
                upperDelay += delayWeights[i * 2 + 1];
                lowerDelay += delayWeights[i * 2];
            }
        }
        return upperDelay - lowerDelay;
    }

public:
    ArbiterPUFModel(int stages, int seed) : numStages(stages) {

```

```

        std::mt19937 rng(seed);
        std::normal_distribution<double> dist(0.0, 1.0);

        delayWeights.resize(numStages * 2);
        for (int i = 0; i < numStages * 2; i++) {
            delayWeights[i] = dist(rng);
        }
    }

    bool evaluate(const std::vector<bool>& challenge) {
        double delay = computePathDelay(challenge);
        return delay > 0.0; // Arbiter decision
    }
};

```

Option 3: Hardware-in-the-Loop (Advanced)

For realistic PUF behavior, interface with actual FPGA-based PUF hardware via serial communication:^{[10][9]}

```

class HardwarePUF {
private:
    SerialPort* fpgaPort;

public:
    HardwarePUF(const char* portName) {
        fpgaPort = new SerialPort(portName, 115200);
    }

    std::vector<uint8_t> evaluate(const std::vector<uint8_t>& challenge) {
        // Send challenge to FPGA
        fpgaPort->write(challenge.data(), challenge.size());

        // Read response
        std::vector<uint8_t> response(16);
        fpgaPort->read(response.data(), 16);

        return response;
    }
};

```

SPONGENT Hash Function Implementation

SPONGENT is a lightweight sponge-based hash function not available in standard C++ crypto libraries. You must implement it from the specification or use a minimal reference implementation.^[^1]

Porting SPONGENT to OMNeT++

```
// SPONGENT.h
#ifndef SPONGENT_H
#define SPONGENT_H

#include <vector>
#include <stdint>

class SPONGENT {
private:
    int rate;           // r = 16 bits for SPONGENT-160
    int capacity;       // c = 144 bits
    int outputSize;     // 160 bits
    std::vector<uint8_t> state; // Internal state (rate + capacity)

    // PRESENT-type permutation (sBox and pLayer)
    void permutation();
    void sBoxLayer();
    void pLayer();

    // Sponge construction phases
    void absorb(const std::vector<uint8_t>& input);
    std::vector<uint8_t> squeeze();

    // PRESENT S-box
    static const uint8_t SBOX[16];

public:
    SPONGENT(int outputBits = 160);

    // Hash function interface
    std::vector<uint8_t> hash(const std::vector<uint8_t>& message);

    // Multi-input hash (for concatenated inputs)
    std::vector<uint8_t> hashMultiple(const std::vector<std::vector<uint8_t>>& inputs);
};

#endif
```

```
// SPONGENT.cc
#include "SPONGENT.h"
#include <string>

const uint8_t SPONGENT::SBOX[16] = {
    0xC, 0x5, 0x6, 0xB, 0x9, 0x0, 0xA, 0xD,
    0x3, 0xE, 0xF, 0x8, 0x4, 0x7, 0x1, 0x2
};

SPONGENT::SPONGENT(int outputBits) : outputSize(outputBits) {
    rate = 16; // SPONGENT-160 uses 16-bit rate
    capacity = 144;
    state.resize((rate + capacity) / 8, 0);
}
```

```

std::vector<uint8_t> SPONGENT::hash(const std::vector<uint8_t>& message) {
    // Initialize state to zero
    std::fill(state.begin(), state.end(), 0);

    // Pad message (10*1 padding)
    std::vector<uint8_t> paddedMsg = message;
    paddedMsg.push_back(0x80); // Append '1' bit
    while ((paddedMsg.size() * 8) % rate != 0) {
        paddedMsg.push_back(0x00); // Pad with zeros
    }

    // Absorb phase
    absorb(paddedMsg);

    // Squeeze phase
    return squeeze();
}

void SPONGENT::absorb(const std::vector<uint8_t>& input) {
    size_t blockSize = rate / 8;
    for (size_t i = 0; i < input.size(); i += blockSize) {
        // XOR input block with state (rate portion)
        for (size_t j = 0; j < blockSize && i + j < input.size(); j++) {
            state[j] ^= input[i + j];
        }

        // Apply permutation
        permutation();
    }
}

std::vector<uint8_t> SPONGENT::squeeze() {
    std::vector<uint8_t> output;
    int bytesNeeded = outputSize / 8;
    int rateBytes = rate / 8;

    while (output.size() < bytesNeeded) {
        // Extract rate portion of state
        for (int i = 0; i < rateBytes && output.size() < bytesNeeded; i++) {
            output.push_back(state[i]);
        }

        if (output.size() < bytesNeeded) {
            permutation();
        }
    }

    return output;
}

void SPONGENT::permutation() {
    // 80 rounds for SPONGENT-160
    for (int round = 0; round < 80; round++) {
        sBoxLayer();
        pLayer();
    }
}

```

```

        // Add round counter (simplified)
        state ^= round;
    }
}

void SPONGENT::sBoxLayer() {
    for (size_t i = 0; i < state.size(); i++) {
        uint8_t byte = state[i];
        uint8_t upper = SBOX[(byte >> 4) & 0x0F];
        uint8_t lower = SBOX[byte & 0x0F];
        state[i] = (upper << 4) | lower;
    }
}

void SPONGENT::pLayer() {
    // Bit permutation (simplified version)
    // Full implementation requires bit-level permutation according to SPONGENT spec
    std::vector<uint8_t> temp = state;
    int totalBits = state.size() * 8;

    for (int i = 0; i < totalBits; i++) {
        int sourceByteIdx = i / 8;
        int sourceBitIdx = i % 8;
        bool bit = (temp[sourceByteIdx] >> sourceBitIdx) & 1;

        // Permutation rule (simplified)
        int targetPos = (i * 31) % totalBits; // Example permutation
        int targetByteIdx = targetPos / 8;
        int targetBitIdx = targetPos % 8;

        if (bit) {
            state[targetByteIdx] |= (1 << targetBitIdx);
        } else {
            state[targetByteIdx] &= ~(1 << targetBitIdx);
        }
    }
}

std::vector<uint8_t> SPONGENT::hashMultiple(const std::vector<std::vector<uint8_t>>& inputs) {
    std::vector<uint8_t> concatenated;
    for (const auto& input : inputs) {
        concatenated.insert(concatenated.end(), input.begin(), input.end());
    }
    return hash(concatenated);
}

```

Note: This is a simplified SPONGENT implementation. For production use, implement the full specification from the original paper or use a verified reference implementation.

Alternative: Using SHA-3 (Keccak) as Substitute

If full SPONGENT implementation is too complex, use SHA-3 (also sponge-based) as a substitute:

```
# Link Crypto++ library
sudo apt-get install libcrypto++-dev
```

```
#include ryctopp/sha3.h>

class HashWrapper {
public:
    static std::vector<uint8_t> hash160(const std::vector<uint8_t>& input) {
        CryptoPP::SHA3_256 sha3;
        std::vector<uint8_t> output(20); // 160-bit
        sha3.CalculateDigest(output.data(), input.data(), input.size());
        output.resize(20); // Truncate to 160 bits
        return output;
    }
};
```

BCH Error Correction Implementation

Use the libcorrect library for BCH encoding/decoding:^{[2][1]}

Installing libcorrect

```
git clone https://github.com/quiet/libcorrect.git
cd libcorrect
mkdir build && cd build
cmake ..
make
sudo make install
```

Integrating BCH in OMNeT++

```
// BCHCodec.h
#ifndef BCHCODEC_H
#define BCHCODEC_H

#include <vector>
#include <stdint>
extern "C" {
    #include rrect.h>
}

class BCHCodec {
private:
    correct_bch* encoder;
    correct_bch* decoder;
```

```

        size_t messageLength;
        size_t parityLength;

public:
    // BCH(255, 131, 18) configuration
    BCHCodec(size_t block_length = 255, size_t data_length = 131,
              size_t min_distance = 18);
    ~BCHCodec();

    // Encode: Generate helper data (parity bits)
    std::vector<uint8_t> encode(const std::vector<uint8_t>& message);

    // Decode: Correct errors using helper data
    std::vector<uint8_t> decode(const std::vector<uint8_t>& received,
                               const std::vector<uint8_t>& helperData);

    // Get number of corrected errors (-1 if too many errors)
    int getErrorCount() const;
};

#endif

```

```

// BCHCodec.cc
#include "BCHCodec.h"
#include <string>
#include <iostream>

BCHCodec::BCHCodec(size_t block_length, size_t data_length, size_t min_distance) {
    // libcorrect uses polynomial representation
    // For BCH(255, 131, 18), use polynomial 0x1f (example)
    encoder = correct_bch_create(8, 0x11d, 18); // GF(2^8), poly, min_distance
    decoder = correct_bch_create(8, 0x11d, 18);

    messageLength = data_length / 8;
    parityLength = (block_length - data_length) / 8;
}

BCHCodec::~~BCHCodec() {
    if (encoder) correct_bch_destroy(encoder);
    if (decoder) correct_bch_destroy(decoder);
}

std::vector<uint8_t> BCHCodec::encode(const std::vector<uint8_t>& message) {
    std::vector<uint8_t> helperData(parityLength);

    // Encode message to generate parity (helper data)
    size_t encodedLen = correct_bch_encode(encoder, message.data(),
                                           message.size(), helperData.data());

    if (encodedLen != parityLength) {
        std::cerr << "BCH encoding error" << std::endl;
    }

    return helperData;
}

```

```

std::vector<uint8_t> BCHCodec::decode(const std::vector<uint8_t>& received,
                                     const std::vector<uint8_t>& helperData) {
    // Concatenate received data with helper data
    std::vector<uint8_t> codeword;
    codeword.insert(codeword.end(), received.begin(), received.end());
    codeword.insert(codeword.end(), helperData.begin(), helperData.end());

    std::vector<uint8_t> corrected(messageLength);

    // Decode with error correction
    ssize_t numErrors = correct_bch_decode(decoder, codeword.data(),
                                           codeword.size(), corrected.data());

    if (numErrors < 0) {
        std::cerr << "BCH decoding failed: too many errors" << std::endl;
    } else {
        std::cout << "BCH corrected " << numErrors << " bit errors" << std::endl;
    }

    return corrected;
}

```

Makefile Configuration

Add libcorrect to your project's Makefile:

```

# In Makefile (generated by opp_makemake)
INCLUDE_PATH += -I/usr/local/include
LIBS += -L/usr/local/lib -lcorrect

```

Or using opp_makemake:

```

opp_makemake -f --deep -I/usr/local/include -L/usr/local/lib -lcorrect

```

Implementing the Four-Phase Protocol

Phase 1: Enrollment Module

Create a C++ simple module for Phase 1 enrollment:

```

// Phase1Enrollment.h
#ifndef PHASE1ENROLLMENT_H
#define PHASE1ENROLLMENT_H

#include <omnetpp.h>
#include "PUFSimulator.h"
#include "BCHCodec.h"
#include <map>

using namespace omnetpp;

```

```

struct CRPRecord {
    std::vector<uint8_t> challenge;
    std::vector<uint8_t> response;
    std::vector<uint8_t> helperData;
};

class Phase1Enrollment : public cSimpleModule {
private:
    // Ground Station database
    std::map<int, std::vector<CRPRecord>> crpDatabase; // UAV_ID -> CRPs
    std::map<int, std::string> uavIdentities; // UAV_ID -> TID

    BCHCodec* bchCodec;
    int numCRPsPerUAV;

protected:
    virtual void initialize() override;
    virtual void handleMessage(cMessage* msg) override;

    // Enrollment operations
    void enrollUAV(int uavId, PUF Simulator* uavPUF);
    void generateCRPs(int uavId, PUF Simulator* uavPUF);
    std::string generateTempID(int uavId);

public:
    // Access to database for other phases
    const std::vector<CRPRecord>& getCRPs(int uavId) const;
};

Define_Module(Phase1Enrollment);

#endif

```

```

// Phase1Enrollment.cc
#include "Phase1Enrollment.h"
#include <sstream>
#include <iomanip>

void Phase1Enrollment::initialize() {
    bchCodec = new BCHCodec(255, 131, 18);
    numCRPsPerUAV = par("numCRPsPerUAV").intValue(); // From omnetpp.ini

    EV << "[Phase 1] Enrollment module initialized" << endl;
}

void Phase1Enrollment::enrollUAV(int uavId, PUF Simulator* uavPUF) {
    EV << "[Phase 1] Enrolling UAV " << uavId << endl;

    // Generate CRPs
    generateCRPs(uavId, uavPUF);

    // Generate temporary identity
    uavIdentities[uavId] = generateTempID(uavId);
}

```

```

    EV << "[Phase 1] UAV " << uavId << " enrolled with "
        << crpDatabase[uavId].size() << " CRPs" << endl;
}

void Phase1Enrollment::generateCRPs(int uavId, PUF Simulator* uavPUF) {
    std::vector<CRPRecord> crps;

    for (int i = 0; i < numCRPsPerUAV; i++) {
        CRPRecord crp;

        // Generate random challenge (128-bit)
        crp.challenge.resize(16);
        for (int j = 0; j < 16; j++) {
            crp.challenge[j] = uniform(0, 255); // OMNeT++ RNG
        }

        // Get PUF response
        crp.response = uavPUF->evaluate(crp.challenge);

        // Generate BCH helper data
        crp.helperData = bchCodec->encode(crp.response);

        crps.push_back(crp);
    }

    crpDatabase[uavId] = crps;
}

std::string Phase1Enrollment::generateTempID(int uavId) {
    std::stringstream ss;
    ss << "TID_" << std::hex << std::setfill('0') << std::setw(8)
        << uniform(0, 0xFFFFFFFF);
    return ss.str();
}

const std::vector<CRPRecord>& Phase1Enrollment::getCRPs(int uavId) const {
    auto it = crpDatabase.find(uavId);
    if (it != crpDatabase.end()) {
        return it->second;
    }
    throw cRuntimeError("UAV %d not enrolled", uavId);
}

void Phase1Enrollment::handleMessage(cMessage* msg) {
    // Handle enrollment requests
    delete msg;
}

```

Phase 2: UAV-GS Authentication

Implement Phase 2 as an application-layer protocol:

```

// Phase2Authentication.h
#ifndef PHASE2AUTHENTICATION_H
#define PHASE2AUTHENTICATION_H

```

```

#include <omnetpp.h>
#include "inet/applications/base/ApplicationBase.h"
#include "inet/common/packet/Packet.h"
#include "PUFSimulator.h"
#include "SPONGENT.h"
#include "BCHCodec.h"
#include "Phase1Enrollment.h"

using namespace omnetpp;
using namespace inet;

// Message types
enum Phase2MsgType {
    AUTH_REQUEST = 1,
    CHALLENGE_ISSUANCE = 2,
    PUF_RESPONSE = 3,
    AUTH_CONFIRMATION = 4
};

// Phase 2 packet structures
struct AuthRequest {
    std::string tempID;
    uint32_t timestamp;
    std::vector<uint8_t> nonce;
    std::vector<uint8_t> authHash;
};

struct ChallengeIssuance {
    std::vector<uint8_t> challenge;
    std::vector<uint8_t> gsNonce;
    std::vector<uint8_t> challengeMAC;
    uint32_t timestamp;
};

struct PUFResponseMsg {
    std::vector<uint8_t> maskedResponse;
    std::vector<uint8_t> authToken;
    std::vector<uint8_t> nextChallenge;
    uint32_t timestamp;
};

class Phase2UAVClient : public ApplicationBase {
private:
    PUF Simulator* puf;
    SPONGENT* hashFunc;
    BCHCodec* bchCodec;

    std::string tempID;
    std::vector<uint8_t> currentNonce;
    simtime_t authStartTime;

    // Statistics
    simsignal_t authLatencySignal;
    simsignal_t commOverheadSignal;

```

```

protected:
    virtual void initialize(int stage) override;
    virtual void handleMessageWhenUp(cMessage* msg) override;

    // Phase 2 steps (UAV side)
    void sendAuthRequest();
    void handleChallengeIssuance(Packet* packet);
    void sendPUFResponse(const std::vector<uint8_t>& challenge,
                        const std::vector<uint8_t>& gsNonce);
    void handleAuthConfirmation(Packet* packet);

public:
    // Performance tracking
    void recordAuthLatency();
    void recordCommOverhead(int bytes);
};

class Phase2GSServer : public ApplicationBase {
private:
    Phase1Enrollment* enrollmentModule;
    SPONGENT* hashFunc;
    BCHCodec* bchCodec;

    std::map<std::string, int> activeAuthentications; // TID -> UAV_ID
    std::map<int, size_t> crpIndex; // UAV_ID -> next CRP index

protected:
    virtual void initialize(int stage) override;
    virtual void handleMessageWhenUp(cMessage* msg) override;

    // Phase 2 steps (GS side)
    void handleAuthRequest(Packet* packet);
    void sendChallengeIssuance(int uavId, const std::string& tempID);
    void handlePUFResponse(Packet* packet);
    void sendAuthConfirmation(int uavId, const std::vector<uint8_t>& sessionKey);

    // Verification
    bool verifyPUFResponse(int uavId, const std::vector<uint8_t>& receivedResponse);
};

Define_Module(Phase2UAVClient);
Define_Module(Phase2GSServer);

#endif

```

```

// Phase2Authentication.cc (UAV Client Implementation)
#include "Phase2Authentication.h"

void Phase2UAVClient::initialize(int stage) {
    ApplicationBase::initialize(stage);

    if (stage == INITSTAGE_LOCAL) {
        int uavId = par("uavId").intValue();
        int pufSeed = par("pufSeed").intValue();
    }
}

```

```

    puf = new PUF Simulator(pufSeed, 0.03, 128);
    hashFunc = new SPONGENT(160);
    bchCodec = new BCHCodec(255, 131, 18);

    tempID = par("tempID").stringValue();

    // Register statistics
    authLatencySignal = registerSignal("authLatency");
    commOverheadSignal = registerSignal("commOverhead");

    EV << "[Phase 2 UAV] Initialized UAV " << uavId << endl;
}

if (stage == INITSTAGE_APPLICATION_LAYER) {
    // Schedule authentication start
    simTime_t startTime = par("authStartTime");
    scheduleAt(simTime() + startTime, new cMessage("startAuth"));
}
}

void Phase2UAVClient::handleMessageWhenUp(cMessage* msg) {
    if (msg->isSelfMessage()) {
        if (strcmp(msg->getName(), "startAuth") == 0) {
            authStartTime = simTime();
            sendAuthRequest();
        }
        delete msg;
    } else {
        Packet* packet = check_and_cast<Packet*>(msg);
        auto msgType = packet->getTag<PacketProtocolTag>()->getProtocol();

        if (msgType == &Protocol::udp) { // Simplified
            // Parse message type from payload
            handleChallengeIssuance(packet); // Step 2 of Phase 2
        }
        delete packet;
    }
}

void Phase2UAVClient::sendAuthRequest() {
    EV << "[Phase 2 UAV] Sending authentication request" << endl;

    // Generate nonce
    currentNonce.resize(16);
    for (int i = 0; i < 16; i++) {
        currentNonce[i] = uniform(0, 255);
    }

    // Create auth hash: H(TID || T1 || N1)
    uint32_t timestamp = static_cast<uint32_t>(simTime().dbl() * 1000);
    std::vector<uint8_t> hashInput;
    hashInput.insert(hashInput.end(), tempID.begin(), tempID.end());
    hashInput.push_back((timestamp >> 24) & 0xFF);
    hashInput.push_back((timestamp >> 16) & 0xFF);
    hashInput.push_back((timestamp >> 8) & 0xFF);
    hashInput.push_back(timestamp & 0xFF);
}

```

```

hashInput.insert(hashInput.end(), currentNonce.begin(), currentNonce.end());

std::vector<uint8_t> authHash = hashFunc->hash(hashInput);

// Create packet
auto packet = new Packet("AuthRequest");
// ... (Add payload with TID, timestamp, nonce, authHash)

// Send to GS
send(packet, "socketOut");

recordCommOverhead(64 + 32 + 128 + 160); // TID + T + N + Hash (bits)
}

void Phase2UAVClient::handleChallengeIssuance(Packet* packet) {
    EV << "[Phase 2 UAV] Received challenge issuance" << endl;

    // Extract challenge, GS nonce, MAC, timestamp from packet
    // ... (Parse packet payload)

    std::vector<uint8_t> challenge; // Extracted from packet
    std::vector<uint8_t> gsNonce;
    // ... verification

    sendPUFResponse(challenge, gsNonce);
}

void Phase2UAVClient::sendPUFResponse(const std::vector<uint8_t>& challenge,
                                     const std::vector<uint8_t>& gsNonce) {
    EV << "[Phase 2 UAV] Generating PUF response" << endl;

    // Evaluate PUF
    std::vector<uint8_t> response = puf->evaluate(challenge);

    // Mask response: R_masked = R XOR H(N2)
    std::vector<uint8_t> hashN2 = hashFunc->hash(gsNonce);
    std::vector<uint8_t> maskedResponse(response.size());
    for (size_t i = 0; i < response.size(); i++) {
        maskedResponse[i] = response[i] ^ hashN2[i];
    }

    // Generate session key: SK = H(R || N1 || N2 || T1)
    std::vector<uint8_t> sessionKeyInput;
    sessionKeyInput.insert(sessionKeyInput.end(), response.begin(), response.end());
    sessionKeyInput.insert(sessionKeyInput.end(), currentNonce.begin(), currentNonce.end());
    sessionKeyInput.insert(sessionKeyInput.end(), gsNonce.begin(), gsNonce.end());
    // ... add timestamp
    std::vector<uint8_t> sessionKey = hashFunc->hash(sessionKeyInput);

    // Generate auth token
    std::vector<uint8_t> authToken = hashFunc->hashMultiple({maskedResponse, sessionKey})

    // Create packet
    auto packet = new Packet("PUFResponse");
    // ... (Add payload)

```

```

        send(packet, "socketOut");

        recordCommOverhead(128 + 160 + 128); // Masked R + Auth Token + Next C
    }

void Phase2UAVClient::recordAuthLatency() {
    simtime_t latency = simTime() - authStartTime;
    emit(authLatencySignal, latency.dbl() * 1000); // Convert to ms
    EV << "[Phase 2 UAV] Authentication latency: " << latency * 1000 << " ms" << endl;
}

void Phase2UAVClient::recordCommOverhead(int bits) {
    emit(commOverheadSignal, bits / 8); // Emit bytes
}

```

Phase 3: UAV-to-UAV Peer Authentication

Implement direct peer authentication:

```

// Phase3PeerAuth.h
#ifndef PHASE3PEERAUTH_H
#define PHASE3PEERAUTH_H

#include <omnetpp.h>
#include "inet/applications/base/ApplicationBase.h"
#include "PUFSimulator.h"
#include "SPONGENT.h"

using namespace omnetpp;
using namespace inet;

struct PeerCredentials {
    int peerUAVId;
    std::vector<uint8_t> challenge;
    std::vector<uint8_t> mask;
};

class Phase3PeerAuthApp : public ApplicationBase {
private:
    PUF Simulator* puf;
    SPONGENT* hashFunc;

    std::map<int, PeerCredentials> peerCredentials; // From Phase 2
    std::vector<uint8_t> ownSecret;                // P_ij

    simsignal_t peerAuthLatencySignal;

protected:
    virtual void initialize(int stage) override;
    virtual void handleMessageWhenUp(cMessage* msg) override;

    // Phase 3 operations
    void authenticateWithPeer(int peerUAVId);
    void sendPeerAuthRequest(int peerUAVId);
    void handleChallengeResponse(Packet* packet);

```

```

void sendMutualAuthCompletion(int peerUAVId, const std::vector<uint8_t>& puzzle);

// Mutual verification
bool verifyPeerPuzzle(const std::vector<uint8_t>& encryptedPuzzle,
                      const std::vector<uint8_t>& peerSecret);

public:
    void loadPeerCredentials(const std::map<int, PeerCredentials>& creds);
};

Define_Module(Phase3PeerAuthApp);

#endif

```

Phase 4: Session Key Establishment

Session key derivation is integrated into Phase 2/Phase 3. For optional ECDH:

```

// Install libsodium for Curve25519
sudo apt-get install libsodium-dev

```

```

#include <sodium.h>

class Phase4SessionKey {
public:
    static std::vector<uint8_t> deriveBasicKey(const std::vector<uint8_t>& pufResponse,
                                                const std::vector<uint8_t>& nonce1,
                                                const std::vector<uint8_t>& nonce2,
                                                SPONGENT* hash) {

        std::vector<uint8_t> input;
        input.insert(input.end(), pufResponse.begin(), pufResponse.end());
        input.insert(input.end(), nonce1.begin(), nonce1.end());
        input.insert(input.end(), nonce2.begin(), nonce2.end());
        return hash->hash(input);
    }

    static std::vector<uint8_t> deriveWithECDH(const std::vector<uint8_t>& basicKey,
                                                uint8_t* publicKeyLocal,
                                                uint8_t* publicKeyRemote) {

        // ECDH key exchange
        uint8_t sharedSecret[crypto_scalarmult_BYTES];
        uint8_t privateKey[crypto_scalarmult_SCALARBYTES];

        // Generate ephemeral key pair
        crypto_box_keypair(publicKeyLocal, privateKey);

        // Compute shared secret
        crypto_scalarmult(sharedSecret, privateKey, publicKeyRemote);

        // Combine with basic key
        std::vector<uint8_t> combined;
        combined.insert(combined.end(), basicKey.begin(), basicKey.end());
        combined.insert(combined.end(), sharedSecret, sharedSecret + crypto_scalarmult_

```

```

        SPONGENT hash(160);
        return hash.hash(combined);
    }
};

```

Network Topology Definition

NED File for UAV Network

Create the network topology in UAVNetwork.ned:

```

// UAVNetwork.ned
package uavauth;

import inet.node.inet.WirelessHost;
import inet.node.inet.Router;
import inet.physicallayer.wireless.ieee80211.packetlevel.Ieee80211ScalarRadioMedium;
import inet.visualizer.integrated.IntegratedCanvasVisualizer;

network UAVAuthNetwork
{
    parameters:
        int numUAVs = default(5);
        @display("bgb=1000,600");

    submodules:
        visualizer: IntegratedCanvasVisualizer {
            @display("p=50,50");
        }

        radioMedium: Ieee80211ScalarRadioMedium {
            @display("p=50,100");
        }

        groundStation: UAVGroundStation {
            @display("p=500,500;i=device/server");
        }

        uav[numUAVs]: UAVNode {
            @display("p=, ,m,5,200,200;i=device/drone");
        }

    connections allowunconnected:
}

// UAV Node with PUF
simple UAVNode extends WirelessHost
{
    parameters:
        @display("i=device/drone");

        // PUF parameters
        int uavId;

```

```

        int pufSeed;
        double pufNoiseLevel = default(0.03);

        // Protocol parameters
        string tempID;
        int numCRPs = default(12);

        // Mobility
        mobility.typename = "LinearMobility";
        mobility.speed = 15mps; // 15 m/s typical UAV speed

        // Applications
        numApps = 2;
        app.typename = "Phase2UAVClient";
        app.uavId = uavId;
        app.pufSeed = pufSeed;
        app.tempID = tempID;

        app[^1].typename = "Phase3PeerAuthApp";
    }

    // Ground Station
    simple UAVGroundStation extends WirelessHost
    {
        parameters:
            @display("i=device/server");

            // Mobility (stationary)
            mobility.typename = "StationaryMobility";

            // Applications
            numApps = 1;
            app.typename = "Phase2GSServer";
            app.numCRPsPerUAV = 12;
    }

```

Configuration File (omnetpp.ini)

```

[General]
network = uavauth.UAVAuthNetwork
sim-time-limit = 200s

# Visualization
*.visualizer*.displayBackground = true
*.visualizer*.displayGrid = true
*.visualizer.mobilityVisualizer.displayMovementTrails = true

# Number of UAVs
*.numUAVs = 5

# UAV Configuration
*.uav[*].uavId = index
*.uav[*].pufSeed = 1000 + index # Unique seed per UAV
*.uav[*].tempID = "TID_" + string(index)

```

```

# Ground Station Position
*.groundStation.mobility.initialX = 500m
*.groundStation.mobility.initialY = 500m

# UAV Mobility
*.uav[*].mobility.initialX = uniform(100m, 900m)
*.uav[*].mobility.initialY = uniform(100m, 400m)
*.uav[*].mobility.initialZ = 50m
*.uav[*].mobility.constraintAreaMinX = 0m
*.uav[*].mobility.constraintAreaMaxX = 1000m
*.uav[*].mobility.constraintAreaMinY = 0m
*.uav[*].mobility.constraintAreaMaxY = 600m

# Wireless Configuration (IEEE 802.11g)
*.radioMedium.backgroundNoise.power = -90dBm
*.uav[*].wlan.radio.transmitter.power = 20mW
*.uav[*].wlan.radio.receiver.sensitivity = -85dBm
*.groundStation.wlan.radio.transmitter.power = 100mW

# Authentication Start Times
*.uav.app.authStartTime = 10s
*.uav[^1].app.authStartTime = 12s
*.uav[^2].app.authStartTime = 14s
*.uav[^3].app.authStartTime = 16s
*.uav[^4].app.authStartTime = 18s

# Peer Authentication (Phase 3)
*.uav[*].app[^1].peerAuthStartTime = 30s + index * 2s

# Statistics Recording
**.authLatency.statistic-recording = true
**.commOverhead.statistic-recording = true
**.vector-recording = true
**.scalar-recording = true

# Output
output-vector-file = results/UAVAuth-${configname}-${runnumber}.vec
output-scalar-file = results/UAVAuth-${configname}-${runnumber}.sca

[Config Baseline]
description = "5 UAVs with basic authentication"

[Config Swarm10]
extends = Baseline
*.numUAVs = 10
description = "10 UAV swarm"

[Config Swarm20]
extends = Baseline
*.numUAVs = 20
description = "20 UAV swarm"

[Config HighNoise]
extends = Baseline
*.uav[*].pufNoiseLevel = 0.05
description = "Test with 5% PUF noise (higher BER)"

```

```
[Config MobilityTest]
extends = Baseline
*.uav[*].mobility.typeName = "RandomWaypointMobility"
*.uav[*].mobility.speed = uniform(10mps, 25mps)
description = "Random waypoint mobility"
```

Result Collection and Analysis

Collecting Statistics

OMNeT++ records statistics automatically when configured in `omnetpp.ini`. Access results programmatically:

```
// In your module (e.g., Phase2UAVClient)
void Phase2UAVClient::finish() {
    recordScalar("totalAuthentications", numAuthentications);
    recordScalar("successRate", (double)successfulAuths / numAuthentications);

    // Calculate average latency from recorded signals
    cStdDev latencyStats;
    // ... (populate from authLatencySignal)
    recordScalar("avgAuthLatency", latencyStats.getMean());
    recordScalar("maxAuthLatency", latencyStats.getMax());
}
```

Analysis Script (Python)

Create a Python script to analyze OMNeT++ result files:

```
# analyze_results.py
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from omnetpp.scave import results, chart, utils

# Load scalar results
df = results.read_result_files('results/*.sca',
                              include_fields_as_scalars=True)

# Filter authentication latency
latency_data = df[df['name'] == 'authLatency:mean']
print(f"Average Authentication Latency: {latency_data['value'].mean():.2f} ms")

# Communication overhead
overhead_data = df[df['name'] == 'commOverhead:sum']
print(f"Total Communication Overhead: {overhead_data['value'].sum()} bytes")

# Success rate
success_data = df[df['name'] == 'successRate']
print(f"Authentication Success Rate: {success_data['value'].mean() * 100:.1f}%")
```

```

# Plot latency distribution
plt.figure(figsize=(10, 6))
plt.hist(latency_data['value'], bins=30, alpha=0.7, edgecolor='black')
plt.xlabel('Authentication Latency (ms)')
plt.ylabel('Frequency')
plt.title('Phase 2 Authentication Latency Distribution')
plt.axvline(latency_data['value'].mean(), color='red', linestyle='--',
            label=f'Mean: {latency_data['value'].mean():.2f} ms')
plt.legend()
plt.savefig('results/latency_distribution.png', dpi=300)
plt.show()

```

Exporting Results to CSV

```

// Custom result export in your simulation
class ResultExporter {
public:
    static void exportToCSV(const std::vector<double>& latencies,
                           const std::vector<int>& overheads,
                           const std::string& filename) {
        std::ofstream file(filename);
        file << "Test,Latency_ms,Overhead_bytes\n";

        for (size_t i = 0; i < latencies.size(); i++) {
            file << "Phase2_" << i << ", "
                << latencies[i] << ", "
                << overheads[i] << "\n";
        }

        file.close();
    }
};

// In finish() method
std::vector<double> latencies;
std::vector<int> overheads;
// ... collect data
ResultExporter::exportToCSV(latencies, overheads, "results/phase2_results.csv");

```

Library Import and Integration Summary

Libraries Required

Library	Purpose	Installation	Integration
libcorrect	BCH error correction	git clone + cmake	Link with -lcorrect
Crypto++ (optional)	Alternative to SPONGENT	apt-get install libcrypto++-dev	Link with -lcryptopp
libsodium (optional)	ECDH for Phase 4	apt-get install libsodium-dev	Link with -lsodium

Library	Purpose	Installation	Integration
INET Framework	Network simulation	git clone + make	Import in NED files

Makefile Configuration

```
# Project Makefile
INCLUDE_PATH = -I$(INET_DIR)/src \
               -I/usr/local/include \
               -I./src/crypto \
               -I./src/nodes \
               -I./src/protocols

LIBS = -L$(INET_DIR)/src -lINET \
       -L/usr/local/lib -lcorrect \
       -lsodium

# Optional: Crypto++
# LIBS += -lcryptopp

SOURCES = $(wildcard src/**/*.cc)
OBJECTS = $(SOURCES:.cc=.o)

# Compile command
%.o: %.cc
    $(CXX) -c $(CXXFLAGS) $(INCLUDE_PATH) $< -o $@

# Link command
UAVAuthProtocol: $(OBJECTS)
    $(CXX) -o $@ $(OBJECTS) $(LIBS)
```

Or using opp_makemake:

```
cd src
opp_makemake -f --deep \
  -I/usr/local/include \
  -L/usr/local/lib \
  -lcorrect -lsodium \
  -KINET_PROJ=../../inet \
  -DINET_IMPORT \
  -I$(INET_PROJ)/src \
  -L$(INET_PROJ)/src -lINET
```

Testing and Debugging

Unit Testing Individual Components

Create test modules for each component:

```
// test_puf.cc
#include "PUFSimulator.h"
#include <iostream>

void testPUFUniqueness() {
    PUF Simulator puf1(42, 0.03);
    PUF Simulator puf2(43, 0.03);

    std::vector<uint8_t> challenge(16, 0xAA);

    auto resp1 = puf1.evaluate(challenge);
    auto resp2 = puf2.evaluate(challenge);

    double uniqueness = PUF Simulator::calculateUniqueness(resp1, resp2);

    std::cout << "Inter-chip Hamming Distance: " << uniqueness << "%" << std::endl;
    assert(uniqueness > 40.0 && uniqueness < 60.0); // Should be ~50%
}

void testPUFReliability() {
    PUF Simulator puf(42, 0.03);
    std::vector<uint8_t> challenge(16, 0xBB);

    std::vector<std::vector<uint8_t>> responses;
    for (int i = 0; i < 10; i++) {
        responses.push_back(puf.evaluate(challenge));
    }

    // Calculate intra-chip Hamming distance
    double avgDistance = 0.0;
    for (size_t i = 1; i < responses.size(); i++) {
        avgDistance += PUF Simulator::calculateUniqueness(responses, responses[i]);
    }
    avgDistance /= (responses.size() - 1);

    std::cout << "Avg Intra-chip Hamming Distance (BER): " << avgDistance << "%" << std::endl;
    assert(avgDistance < 5.0); // Should be close to noise level (3%)
}

int main() {
    testPUFUniqueness();
    testPUFReliability();
    std::cout << "All PUF tests passed!" << std::endl;
    return 0;
}
```

Compile and run:

```
g++ -std=c++11 test_puf.cc src/crypto/PUFSimulator.cc -I./src/crypto -o test_puf
```

```
./test_puf
```

Debugging in OMNeT++ IDE

1. Enable Debug Mode:

```
make MODE=debug
```

2. Set Breakpoints in OMNeT++ IDE at critical points:

- PUF evaluation in `Phase2UAVClient::sendPUFResponse()`
- BCH decoding in `Phase2GSServer::verifyPUFResponse()`
- Session key derivation

3. Use EV Logging:

```
EV_DEBUG << "[Phase 2] Challenge: " << challengeHex << endl;  
EV_INFO << "[Phase 2] Authentication successful" << endl;  
EV_ERROR << "[Phase 2] BCH decoding failed" << endl;
```

4. Inspect Simulation State via Qtenv inspector:

- Right-click on module → Inspect
- View internal variables, messages in transit
- Monitor signal emissions

Performance Validation

Comparing with Python Implementation

Export metrics from your Python simulation:

```
# In your Python code (from UAV-Authentication repo)  
import csv  
  
with open('python_results.csv', 'w', newline='') as f:  
    writer = csv.writer(f)  
    writer.writerow(['Phase', 'Latency_ms', 'Overhead_bytes'])  
    writer.writerow(['Phase2', phase2_latency * 1000, phase2_overhead])  
    writer.writerow(['Phase3', phase3_latency * 1000, phase3_overhead])
```

Compare with OMNeT++ results:

```
# compare_results.py  
import pandas as pd  
  
python_df = pd.read_csv('python_results.csv')  
omnet_df = pd.read_csv('results/phase2_results.csv')
```

```
print("Python Implementation:")
print(python_df.describe())

print("\nOMNeT++ Implementation:")
print(omnet_df.describe())

print("\nDifference (OMNeT++ vs Python):")
print(f"Latency: {omnet_df['Latency_ms'].mean() - python_df['Latency_ms'].mean():.2f} ms'
print(f"Overhead: {omnet_df['Overhead_bytes'].mean() - python_df['Overhead_bytes'].mean
```

Target Performance Metrics

Based on your protocol specification:^{[1][2]}

Metric	Target	Measurement
Phase 2 Latency	10-50 ms	Record time from sendAuthRequest() to handleAuthConfirmation()
Phase 3 Latency	0.8-1.2 ms	Record time for peer authentication (no GS)
Phase 2 Overhead	210 bytes	Sum of 4 message sizes
Phase 3 Overhead	196 bytes	Sum of 3 message sizes
Success Rate	>98%	Track successful authentications / total attempts
BCH Correction	Up to 18 errors	Monitor bchCodec->getErrorCount()

Advanced Topics

Integrating with UAV Mobility Models

Use INET's built-in mobility models or custom UAV flight patterns:^{[12][13]}

```
// Advanced UAV mobility
*.uav[*].mobility.typename = "MassMobility"
*.uav[*].mobility.changeInterval = truncnormal(2s, 0.5s)
*.uav[*].mobility.changeAngleBy = normal(0deg, 30deg)
*.uav[*].mobility.speed = truncnormal(15mps, 5mps)
*.uav[*].mobility.updateInterval = 100ms
```

Or custom waypoint mission:

```
class UAVMissionMobility : public MobilityBase {
private:
    std::vector<Coord> waypoints;
    size_t currentWaypointIdx;

protected:
```

```

    virtual void setTargetPosition() override {
        targetPosition = waypoints[currentWaypointIdx];
        currentWaypointIdx = (currentWaypointIdx + 1) % waypoints.size();
    }

public:
    void loadMission(const std::vector<Coord>& mission) {
        waypoints = mission;
    }
};

```

Simulating Attack Scenarios

Test protocol security by simulating attacks:^{[2][1]}

```

// MaliciousUAV.cc - Simulates replay attack
class MaliciousUAV : public UAVNode {
protected:
    std::vector<Packet*> capturedPackets;

    virtual void handleMessageWhenUp(cMessage* msg) override {
        Packet* packet = dynamic_cast<Packet*>(msg);

        if (packet && uniform(0, 1) < 0.1) { // 10% chance to capture
            // Store packet for replay
            capturedPackets.push_back(packet->dup());
            EV << "[Malicious] Captured packet for replay" << endl;
        }

        // Replay attack after delay
        if (!capturedPackets.empty() && uniform(0, 1) < 0.05) {
            send(capturedPackets->dup(), "socketOut");
            EV << "[Malicious] Replaying captured packet" << endl;
        }

        UAVNode::handleMessageWhenUp(msg);
    }
};

```

Test man-in-the-middle:

```

class MITMAttacker : public cSimpleModule {
protected:
    virtual void handleMessage(cMessage* msg) override {
        Packet* packet = check_and_cast<Packet*>(msg);

        // Attempt to modify packet
        // ... tampering logic

        // Forward modified packet
        send(packet, "out");

        EV << "[MITM] Forwarded tampered packet" << endl;
    }
};

```

```
}  
};
```

Hardware-in-the-Loop Testing

For ultimate realism, interface OMNeT++ with real hardware:^{[14][9]}

```
class HardwarePUFInterface {  
private:  
    SerialPort* fpgaConnection;  
  
public:  
    HardwarePUFInterface(const std::string& port) {  
        fpgaConnection = new SerialPort(port, 115200);  
    }  
  
    std::vector<uint8_t> evaluatePUF(const std::vector<uint8_t>& challenge) {  
        // Send challenge to FPGA via UART  
        fpgaConnection->write(challenge.data(), challenge.size());  
  
        // Wait for response  
        std::vector<uint8_t> response(16);  
        fpgaConnection->read(response.data(), 16, 100); // 100ms timeout  
  
        return response;  
    }  
};
```

Enable in omnetpp.ini:

```
[Config Hardware]  
*.uav.app.pufType = "hardware"  
*.uav.app.hardwarePUFPort = "/dev/ttyUSB0"
```

Troubleshooting Common Issues

Issue: BCH Decoding Always Fails

Cause: PUF noise level too high for BCH correction capability.

Solution:

1. Reduce PUF noise: `*.uav[*].pufNoiseLevel = 0.02`
2. Use stronger BCH code: `BCHCodec(511, 259, 36)` (corrects 36 errors)
3. Verify helper data stored correctly during enrollment

Issue: Authentication Timeouts

Cause: Network delays exceed timestamp validation window.

Solution:

```
# Increase timestamp window
*.uav[*].app.timestampWindow = 5s # Instead of 2s

# Reduce network latency
*.uav[*].wlan.radio.propagationDelay = 0.1ms
```

Issue: PUF Responses Not Consistent

Cause: RNG seed not set correctly, noise model incorrect.

Solution:

```
// Ensure deterministic seed per UAV
PUFSimulator::PUFSimulator(int seed, ...) {
    pufRng.seed(seed); // Must be unique per UAV
    noiseRng.seed(seed + 10000); // Different seed for noise
}
```

Issue: OMNeT++ Crashes on Library Link

Cause: Missing library paths or version mismatch.

Solution:

```
# Check library installation
ldconfig -p | grep correct
ldconfig -p | grep sodium

# If missing, add to LD_LIBRARY_PATH
export LD_LIBRARY_PATH=/usr/local/lib:$LD_LIBRARY_PATH

# Recompile with verbose output
make MODE=release V=1
```

Conclusion

This guide provides a complete roadmap for implementing your PUF-Based UAV Authentication Protocol in OMNeT++. While no native PUF or SPONGENT libraries exist for OMNeT++, the combination of custom C++ implementations, external libraries (libcorrect, libsodium), and INET Framework integration enables realistic simulation of all four protocol phases.^{[4][5][3][1][^2]}

Key Implementation Steps:

1. Set up OMNeT++ 6.0+ with INET Framework
2. Implement PUF simulator (software model or hardware interface)
3. Port SPONGENT hash or use SHA-3 substitute
4. Integrate libcorrect for BCH error correction
5. Implement four-phase protocol as OMNeT++ modules
6. Define UAV network topology in NED files
7. Configure simulations in omnetpp.ini
8. Collect and analyze results with built-in statistics

Next Steps:

- Test baseline 5-UAV scenario
- Validate against your Python implementation metrics
- Scale to 20+ UAV swarms
- Simulate attack scenarios (replay, MITM)
- Export results for publication/presentation

For further assistance, consult the [OMNeT++ documentation](#), your [GitHub repository](#), and the implementation guide in your attached files. Your Python implementation serves as an excellent reference for protocol logic and expected performance baselines.^{[3][14][6][2]}

References

1. [PUF-Based-UAV-Authentication-Protocol-Complete-Four-Phase-Architecture.pdf](#) - This protocol provides secure authentication for UAV swarm networks using Physical Unclonable Functions...
2. [IMPLEMENTATION_GUIDE.md](#) - # COMPLETE IMPLEMENTATION GUIDE

PUF-Based UAV Authentication Protocol Simulation

For: IIITG ...

3. [PUF-Based-UAV-Authentication.pdf](#) - Physical Unclonable Functions (PUFs) provide lightweight, hardware-based authentication for resource...
4. [Steps to Implement Physical Layer in OMNeT++](#) - How to Implement Physical Layer in OMNeT++ · Step 1: Set Up OMNeT++ and INET Framework · Step 2: Cre...
5. [Steps to Implement Cryptography in OMNeT++](#) - Install OMNeT++ and INET Framework · Create a New OMNeT++ Project · Define the Network Topology · Co...
6. [OMNeT++ Discrete Event Simulator](#) - OMNeT++ is an extensible, modular, component-based C++ simulation library and framework, primarily f...

7. INET Framework - INET Framework is an open-source model library for the OMNeT++ simulation environment. It provides p...
8. INET Framework - What Is INET Framework? - INET Framework is an open-source model library for the OMNeT++ simulation environment. It provides p...
9. Design and Implementation of Cost-Effective End-to-End Authentication Protocol for PUF-Enabled IoT Devices - The ubiquitous presence of Internet of Things (IoT) prospers in every aspect of human life. The low-...
10. Shift Register, Reconvergent-Fanout (SiRF) PUF Implementation on an FPGA - Physical unclonable functions (PUFs) are gaining traction as an attractive alternative to generating...
11. A Novel FPGA Implementation of the NAND-PUF with Minimal Resource Usage and High Reliability - In this work we propose a novel implementation on recent Xilinx FPGA platforms of a PUF architecture...
12. Steps to implement UAV based VANET in OMNeT++ - Install OMNeT++ and INET Framework · Create a New OMNeT++ Project · Import INET into Your Project · ...
13. Steps to Start Drone based VANET Projects using OMNeT++ - In this given procedure, we had widely presented the instructions regarding the implementation of Dr...
14. Porting Real-World Protocol Implementations into OMNeT++ - In this blog post, we explore some of the workable approaches of bringing an existing protocol imple...
15. Selection of SRAM Cells to improve Reliable PUF implementation using Cell Mismatch Metric - Physically Unclonable Functions (PUFs) are low-cost cryptographic primitives implemented in secret k...